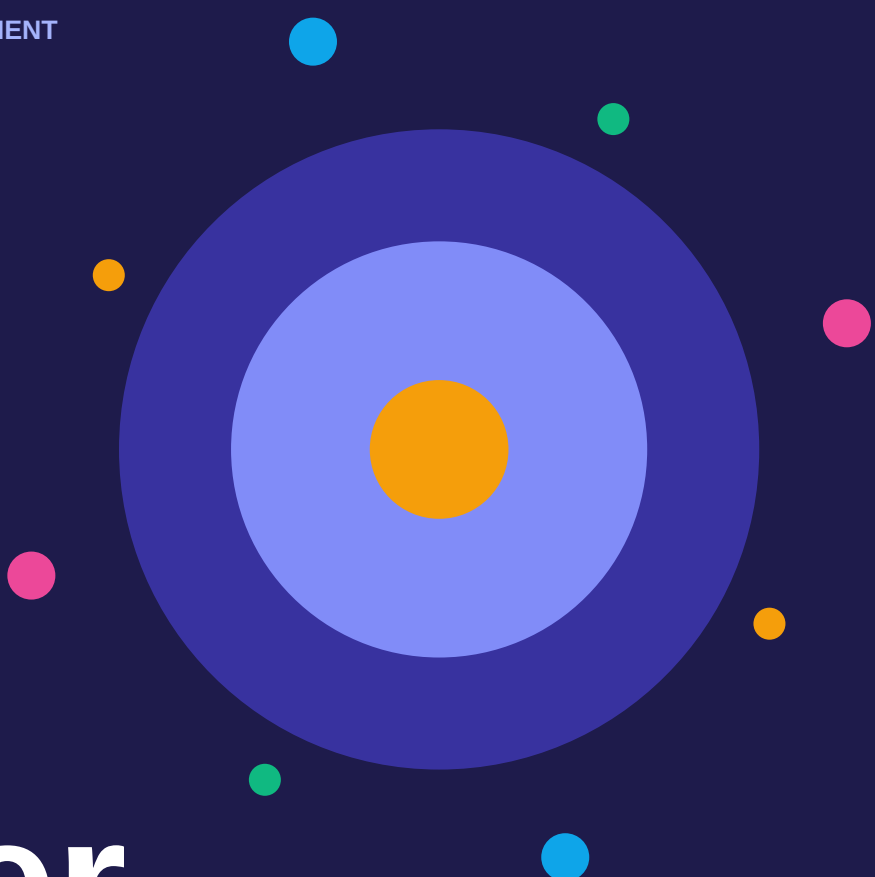




Gather

Real-time chatrooms and private conversations,
all in one place.

VERSION	1.0 (Draft for review)
DATE	21 September 2026
PLATFORMS	Responsive web app / installable PWA
TECH STACK	React (TypeScript) + ASP.NET Core (.NET) + SignalR
OWNER	Product Owner - to be assigned

Contents

1. Product Overview

2. Scope and Release Plan

3. Functional Requirements

4. Additional Recommended Features

5. Non-Functional Requirements

6. Technical Architecture

7. Security and Privacy Requirements

8. UX and Design

9. Success Metrics

10. Roadmap and Milestones

11. Risks and Mitigations

12. Open Questions

Appendix A. Sample Acceptance Criteria

3

4

4

8

10

11

15

16

18

18

19

19

20

Document control

Version	Date	Author	Notes
0.1	15 Sep 2026	Product	Initial concept and scope
1.0	21 Sep 2026	Product	Full PRD draft for engineering and design review

How to read this document
Requirements carry an ID and a priority. **P0** = required for the v1.0 launch, **P1** = planned for v1.1, **P2** = future / v2. Numbers and targets marked as proposals should be validated with the team before they are locked.

1. Product Overview

1.1 Summary

Gather is a real-time chat platform where people create **chatrooms** around a topic, club, class or friend group and talk in them using text, images and videos. Any user can also start a private **one-on-one conversation** with another person simply by searching for their **username**. It is built as a responsive React web app on top of an ASP.NET Core backend that uses SignalR for real-time delivery.

The product name reflects its core idea: making it effortless for people to gather - in a shared room, or one to one - without the setup overhead and feature sprawl of larger community platforms.

1.2 Problem statement

Small communities, student groups, hobby circles and project teams need one place that combines group rooms and private messaging. Today they typically stitch together several apps, or adopt a large platform whose complexity, noise and setup cost is out of proportion to what a group of 5 to 500 people needs. They want to create a room in seconds, invite people with a link, share photos and clips, and message individuals directly - and have it feel fast and simple.

1.3 Goals

- **Instant rooms:** a new user can sign up, create a room and invite others in under 2 minutes.
- **Truly real-time:** messages, typing indicators and presence feel instantaneous (p95 delivery under 300 ms, same region).
- **Rich but safe sharing:** images and videos are first-class, with validation, scanning and moderation built in from day one.
- **Frictionless 1:1 chat:** find anyone by username and start talking, with strong controls to block and limit who can reach you.
- **Solid foundation:** an architecture that scales from hundreds to tens of thousands of concurrent users without a rewrite.

1.4 Non-goals for v1.0

- Voice and video calling, screen share and live streaming (planned for v2).
- Native iOS and Android apps (the PWA covers mobile in v1; React Native is a v2 option).
- End-to-end encryption (transport and at-rest encryption only in v1).
- Bots, public API for third-party developers, and monetisation features.
- Federation or interoperability with other chat networks.

1.5 Target users and personas

Persona	Profile	What they need from Gather
Aarav - the Community Creator	24, runs a college club with 150 members. Organises events and shares photos and clips.	Create and customise a room quickly, invite by link, manage members and roles, keep spam out, share media.
Meera - the Active Member	20, in several groups. Chats mostly on her phone.	Fast mobile web experience, reliable notifications, easy media sharing, DM friends by username.

Persona	Profile	What they need from Gather
Kabir - the Project Lead	29, coordinates a small remote team of 10.	Organised rooms, pinned messages, search, file sharing, mentions, low-noise notification controls.

1.6 Key terminology

Term	Definition
Room	A shared space with members, roles and a message history. Public (discoverable) or private (invite-only).
Channel	A message stream. Every room has a default #general channel. Direct conversations are also channels of type 'Direct'. Multiple channels per room are exposed in the UI from v1.1.
DM	A private one-to-one conversation between two users, started by searching a username.
Username	A unique, case-insensitive handle (for example aarav.k) used to find and mention people.
Member	A user who has joined a room. Members have a role: Owner, Admin, Moderator or Member.

2. Scope and Release Plan

Scope is phased so that the core promise - rooms, real-time text, media and DMs - ships first and the extra features from section 4 land in follow-up releases without blocking launch.

Release	Theme	Included
v1.0 (MVP)	Core chat	Sign-up and login, profiles, create/join rooms, invite links, roles (Owner/Admin/Member), real-time messaging, history with infinite scroll, typing and presence, unread counts, image/video/file sharing, 1:1 DMs by username, block, kick/ban, rate limiting, light/dark theme, responsive layout.
v1.1	Engagement and control	Multiple channels per room, replies, emoji reactions, @mentions, pinned messages, message search, Markdown-lite formatting, GIF picker, web push and email notifications, notification levels, report and review queue, friends list, message requests, PWA install, social login.
v1.2	Polish	Voice messages, media gallery, custom roles and permissions, room templates, session management, audit log UI, Ctrl+K quick switcher.
v2.0	Expansion	Voice and video calls (WebRTC), screen share, threads, group DMs, polls, link previews, bots and webhooks, scheduled messages, E2EE for DMs (evaluate), video transcoding to HLS, React Native apps, i18n.

3. Functional Requirements

Each requirement has a unique ID. Priorities: **P0** launch-blocking, **P1** v1.1, **P2** later.

3.1 Accounts and authentication (AUTH)

ID	Requirement	Priority
AUTH-01	Register with email, username and password. Password minimum 10 characters, checked against common-password lists.	P0
AUTH-02	Username rules: 3-20 characters; letters, numbers, underscore and dot; case-insensitive unique; reserved words blocked. Can be changed once every 30 days.	P0

ID	Requirement	Priority
AUTH-03	Log in with email or username plus password. Short-lived JWT access token (15 min) with a rotating refresh token in an httpOnly, Secure, SameSite cookie.	P0
AUTH-04	Email verification and password reset through signed, single-use, expiring links. Responses must not reveal whether an account exists.	P0
AUTH-05	Profile: display name, avatar upload, short bio. Account lockout with back-off after repeated failed logins.	P0
AUTH-06	Presence status: Online, Idle (auto), Do Not Disturb and Invisible, with optional custom status text.	P1
AUTH-07	Social login (Google, GitHub) and account linking.	P1
AUTH-08	Active sessions list with the ability to revoke any device. Account deletion and personal data export.	P1
AUTH-09	Two-factor authentication (TOTP) and passkeys.	P2

3.2 Chatrooms (ROOM)

ID	Requirement	Priority
ROOM-01	Create a room with name, description, icon and visibility (Public or Private). The creator becomes Owner. A default #general channel is created automatically.	P0
ROOM-02	Join public rooms from a Discover page (browse and search) and private rooms only through an invite.	P0
ROOM-03	Invite links: generate, choose expiry (1 hour, 1 day, 7 days, never) and max uses, copy and revoke. Opening a link while logged out routes through sign-up, then joins the room.	P0
ROOM-04	Roles: Owner, Admin, Moderator, Member, with the permission matrix below. Owner can transfer ownership.	P0
ROOM-05	Member list with role badges and live presence. Search members within a room.	P0
ROOM-06	Leave room, delete room (Owner only, typed-name confirmation), and edit room name, icon and description.	P0
ROOM-07	Room size cap of 1,000 members in v1.0 (proposal; revisit after load testing).	P0
ROOM-08	Room settings: slow mode, restrict who can post media, member limit, join approval for public rooms.	P1
ROOM-09	Multiple text channels and categories inside a room, with per-channel permissions.	P1
ROOM-10	Discovery tags and categories, featured rooms, and room templates (Study group, Club, Friends).	P1
ROOM-11	Custom roles with granular permissions.	P2

Room permission matrix (v1.0)

Action	Owner	Admin	Moderator	Member
Read and send messages, upload media	Yes	Yes	Yes	Yes
Edit / delete own messages	Yes	Yes	Yes	Yes
Delete other members' messages	Yes	Yes	Yes	No
Pin messages (v1.1)	Yes	Yes	Yes	No
Mute or kick members	Yes	Yes	Yes	No
Ban / unban members	Yes	Yes	No	No
Create and revoke invite links	Yes	Yes	No	No
Edit room settings and manage roles	Yes	Yes (not other Admins)	No	No
Delete room / transfer ownership	Yes	No	No	No

3.3 Real-time messaging (MSG)

ID	Requirement	Priority
MSG-01	Send and receive text messages in real time in rooms and DMs. Max 4,000 characters per message.	P0
MSG-02	Persistent history with cursor-based pagination and infinite scroll upwards. The message list must be virtualised for long histories.	P0
MSG-03	Optimistic sending with Pending / Sent / Failed states and one-tap retry. A client-generated message ID makes sends idempotent (no duplicates on retry).	P0
MSG-04	Edit and delete own messages ('edited' label; soft delete). Moderators and above can delete others' messages.	P0
MSG-05	Typing indicators and presence (online, idle, offline) scoped to rooms and DMs the user is viewing.	P0
MSG-06	Unread counts per room and DM, last-read marker, and a 'jump to first unread' control.	P0
MSG-07	Automatic reconnect with backoff and gap-fill: after a drop the client fetches messages newer than the last one it holds.	P0
MSG-08	Markdown-lite (bold, italic, inline code, code block, auto-linked URLs) and an emoji picker.	P1
MSG-09	Reply to a message with a quoted preview; jump to the original.	P1
MSG-10	Emoji reactions with counts and who-reacted tooltip.	P1
MSG-11	@username mentions with highlight and notification; @everyone / @here restricted by role.	P1
MSG-12	Pinned messages per channel.	P1
MSG-13	Full-text message search within a room or DM, filterable by sender, date and 'has media'.	P1
MSG-14	Threads, link previews (server-side unfurl with SSRF protection), polls, scheduled messages.	P2

3.4 Media sharing (MEDIA)

ID	Requirement	Priority
MEDIA-01	Share images (JPEG, PNG, GIF, WebP) up to 10 MB with inline preview and a lightbox viewer.	P0
MEDIA-02	Share videos (MP4, WebM, MOV) up to 100 MB (configurable) with an inline player, poster frame and native controls.	P0
MEDIA-03	Up to 10 attachments per message. Drag-and-drop, paste from clipboard, per-file progress and cancel, caption text.	P0
MEDIA-04	Server validates every upload: extension allow-list, MIME sniffing by magic bytes, size limits. Images are re-encoded and EXIF/GPS metadata stripped.	P0
MEDIA-05	Uploads go directly to object storage through pre-signed URLs (S3) or SAS tokens (Azure Blob). Media is delivered via CDN; private rooms and DMs use short-lived signed URLs.	P0
MEDIA-06	Thumbnails for images and a poster frame for videos are generated asynchronously; the UI shows a placeholder until ready.	P0
MEDIA-07	General file sharing (PDF, Office documents, zip) up to 25 MB, shown as download cards.	P1
MEDIA-08	Malware scanning of uploaded files (for example ClamAV) before they become downloadable.	P1
MEDIA-09	Per-user storage quota, per-room media gallery, and retention policy for old media.	P1
MEDIA-10	Video transcoding to H.264/AAC and adaptive streaming (HLS) for large videos.	P2

3.5 Direct messages (DM)

ID	Requirement	Priority
DM-01	Search users by username (prefix match, minimum 3 characters). Results show avatar, display name and username. Search is rate-limited to deter scraping.	P0
DM-02	Start a 1:1 conversation from search results, a profile card or the member list. Exactly one conversation exists per pair of users.	P0
DM-03	DM inbox sorted by latest activity with unread badges. DMs support all P0 messaging and media features.	P0
DM-04	Block and unblock users. A blocked user cannot send DMs, mention or see presence of the blocker.	P0
DM-05	Privacy setting: who may DM me - Everyone, Members of shared rooms, or Friends only.	P1
DM-06	Message requests: first message from a non-friend lands in a Requests folder to accept or decline.	P1
DM-07	Friend requests and friends list.	P1
DM-08	Group DMs (3-10 people) and DM read receipts (opt-out).	P2

3.6 Notifications (NOTIF)

ID	Requirement	Priority
NOTIF-01	In-app unread badges, mention highlights and toast notifications for events outside the current view.	P0
NOTIF-02	Web Push notifications (service worker). On iOS these require the PWA to be installed to the home screen.	P1
NOTIF-03	Email digest for missed DMs and mentions, respecting unsubscribe and frequency preferences.	P1

ID	Requirement	Priority
NOTIF-04	Per-room and per-DM notification level (All / Mentions only / Nothing) and mute for a duration.	P1
NOTIF-05	Do Not Disturb schedules.	P2

3.7 Moderation and safety (MOD)

ID	Requirement	Priority
MOD-01	Kick and ban members with an optional reason; banned users cannot rejoin via invite. Unban supported.	P0
MOD-02	Rate limiting and anti-spam: messages per second, uploads per minute, invites per hour, DM starts per day, per user and per IP.	P0
MOD-03	Terms of Service, community guidelines and a minimum age gate (13+; higher where local law requires).	P0
MOD-04	Report a message, user or room with a reason; reports reach room moderators and the platform review queue.	P1
MOD-05	Timed mute (timeout) for members. Room audit log of moderation actions.	P1
MOD-06	Platform admin console: user and room lookup, takedown, global ban, content review.	P1
MOD-07	Auto-moderation rules (blocked words, link and invite filtering, raid protection).	P2

3.8 Experience and accessibility (UX)

ID	Requirement	Priority
UX-01	Responsive layout from 360 px phones to wide desktops, with light and dark themes following system preference.	P0
UX-02	WCAG 2.1 AA: keyboard navigation, screen-reader labels and live regions for new messages, sufficient contrast, reduced-motion support.	P0
UX-03	Empty, loading (skeleton), error and offline states for every screen. Connection status banner when the socket is down.	P0
UX-04	Installable PWA with offline shell.	P1
UX-05	Keyboard shortcuts and Ctrl+K quick switcher for rooms, DMs and people.	P1
UX-06	Localisation (i18n) and right-to-left layout support.	P2

4. Additional Recommended Features

Beyond the requested scope, these features have the strongest impact on engagement, safety and retention for a Discord-style product. Effort is a rough size: **S** (days), **M** (1-2 weeks), **L** (multi-week).

Feature	Why it matters	Effort	Release
Emoji reactions	Lightweight interaction; reduces 'noise' replies such as 'ok' and '+1'.	S	v1.1
Replies and @mentions	Keeps busy rooms readable and makes sure people notice what concerns them.	M	v1.1
Message search	Becomes essential once a room has more than a few days of history.	M	v1.1
Pinned messages	Rules, links and announcements stay visible.	S	v1.1

Feature	Why it matters	Effort	Release
Notification levels and mute	The main defence against notification fatigue; directly improves retention.	M	v1.1
Web Push and PWA install	Brings people back on mobile without a native app.	M	v1.1
Report and review queue	Needed for trust and safety once media sharing is public.	M	v1.1
GIF picker	Popular for casual chat, low effort through a third-party GIF provider.	S	v1.1
Friends list and message requests	Gives users control over who can contact them by username.	M	v1.1
Voice messages	Fast to record on phones; a good stepping stone before live calls.	M	v1.2
Room templates and onboarding	Faster first-room setup improves activation.	S	v1.2
Voice and video calls, screen share	The feature people most associate with Discord; built on WebRTC with an SFU (for example LiveKit).	L	v2
Threads and polls	Structure for larger communities and group decisions.	M	v2
Bots, webhooks, slash commands	Ecosystem growth: moderation bots, integrations, notifications from other tools.	L	v2
End-to-end encrypted DMs	Strong privacy differentiator, but complicates search, multi-device and moderation; evaluate after v1.	L	v2+

5. Non-Functional Requirements

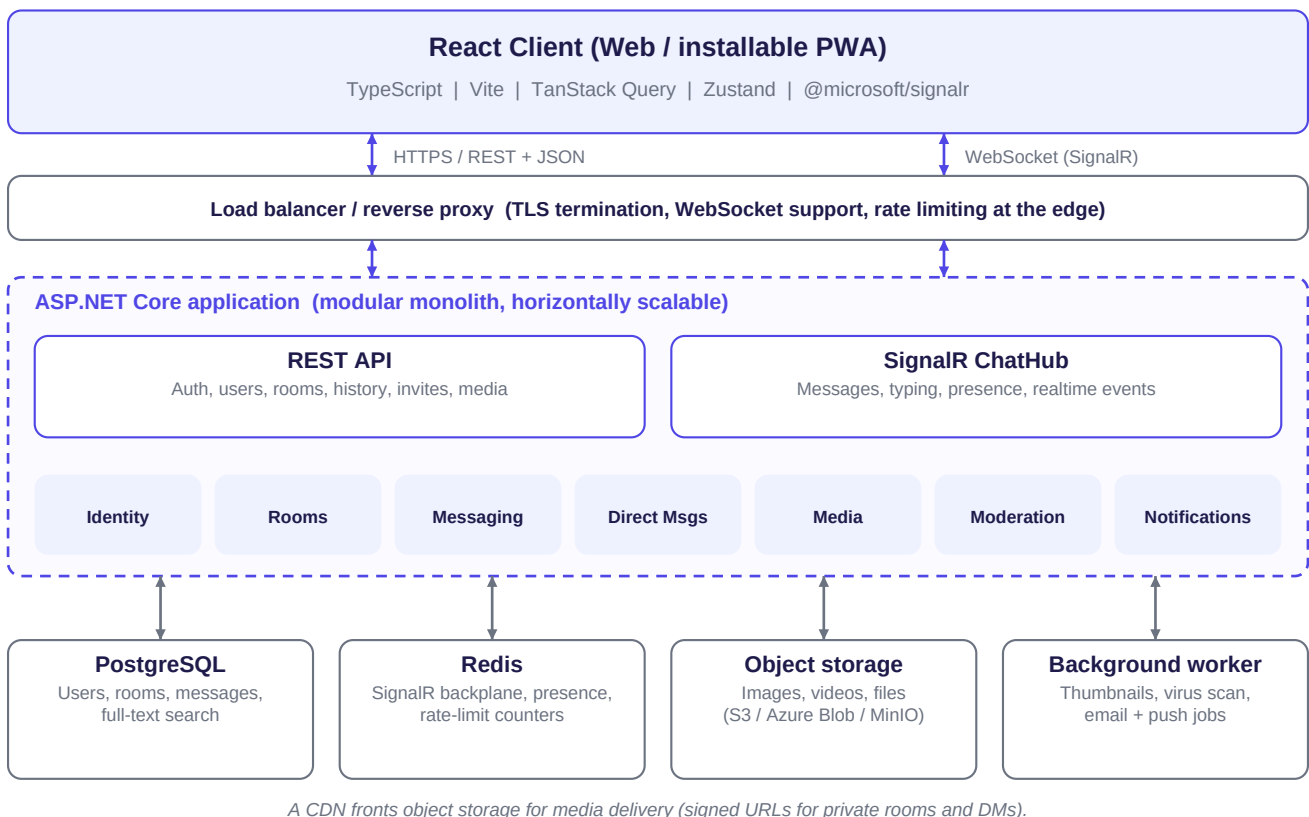
Numeric targets are proposals for the first launch and should be confirmed through load testing.

Category	Requirement
Performance	p95 message delivery under 300 ms within one region. REST p95 under 200 ms. Largest Contentful Paint under 2.5 s on a mid-range phone on 4G. Initial JS bundle under 250 KB gzipped via code splitting.
Scalability	Launch target: 10,000 concurrent connections across 2+ app instances. SignalR scale-out via Redis backplane (or Azure SignalR Service). Stateless API nodes; no in-process state that prevents horizontal scaling.
Availability	99.5% monthly at launch, 99.9% target within 6 months. Zero-downtime deploys with graceful connection draining.
Durability	Automated daily database backups with point-in-time recovery; tested restore procedure. Media on replicated object storage.
Security	OWASP ASVS Level 2 as the baseline: TLS 1.2+, HSTS, CSP, strict CORS allow-list, CSRF protection for cookie flows, dependency and container scanning in CI. See section 7.
Privacy and compliance	Privacy by design: data minimisation, user data export and deletion, consent for optional emails and push. Aligned with GDPR and India's DPDP Act 2023; legal review required before launch in each region.
Accessibility	WCAG 2.1 AA on all core flows, verified with automated (axe) and manual screen-reader testing.
Browser support	Latest two versions of Chrome, Edge, Firefox and Safari; iOS Safari 16.4+ and Chrome for Android.
Observability	Structured logs (Serilog), metrics and distributed traces (OpenTelemetry), dashboards for connection count, message rate, error rate and queue depth; alerting on SLO burn.
Quality	Unit tests for domain and services (70%+ coverage target), integration tests with Testcontainers, E2E with Playwright, load tests before launch, CI/CD with preview environments.

6. Technical Architecture

6.1 System architecture

Gather starts as a **modular monolith**: one deployable ASP.NET Core application with clear internal modules (Identity, Rooms, Messaging, Direct Messages, Media, Moderation, Notifications). This keeps development and operations simple for a small team while leaving clean seams to extract services later - most likely Media processing and Notifications first.



6.2 Technology stack

Layer	Recommendation	Notes
Frontend	React 18/19 + TypeScript, Vite	TanStack Query for server state, Zustand (or Redux Toolkit) for UI/realtime state, Tailwind CSS with Radix UI primitives, React Hook Form + Zod, react-virtuoso for the message list, Vitest + Testing Library + Playwright.
Realtime client	@microsoft/signalr	WebSockets with automatic reconnect; falls back to SSE / long polling when needed.
Backend	ASP.NET Core on .NET (10 LTS recommended, 8 LTS acceptable)	Minimal APIs or controllers, FluentValidation, EF Core, Serilog, built-in rate limiting middleware. Modular monolith following clean-architecture boundaries.
Realtime server	ASP.NET Core SignalR	Groups per room/channel and per user. Redis backplane for multi-node fan-out.
Auth	ASP.NET Core Identity + JWT bearer	Refresh-token rotation with reuse detection. SignalR authenticates via the access token on the WebSocket handshake.

Layer	Recommendation	Notes
Database	PostgreSQL 16+ (Npgsql)	Strong indexing and built-in full-text search for MVP. SQL Server is a workable alternative if the team standardises on it.
Cache and presence	Redis	Backplane, presence heartbeats, typing state, rate-limit counters, short-lived caches.
Object storage and CDN	S3-compatible (AWS S3, Cloudflare R2, MiniIO for local dev) or Azure Blob	Pre-signed uploads, CDN delivery, lifecycle rules.
Background jobs	Hangfire or Quartz.NET (or Azure Functions / queue workers)	Thumbnails, EXIF stripping, malware scan, email and push delivery, cleanup jobs.
Media processing	libvips / SkiaSharp / ImageSharp, FFmpeg	Check library licences for commercial use before choosing.
Infrastructure	Docker, GitHub Actions, Terraform or Bicep	Azure Container Apps / App Service or AWS ECS / Kubernetes. Separate dev, staging and production.

6.3 Real-time design

The ChatHub SignalR hub is the real-time entry point. Each authenticated connection joins a personal group `user:{userId}` (unread counters, DM notifications, friend and block events) and subscribes to `channel:{channelId}` groups only for the channels the user currently has open. Subscribing lazily keeps fan-out small for users who belong to many rooms.

Send-message flow

- 1 Client creates a client message ID (UUID), renders the message optimistically as 'Pending', and invokes `SendMessage` over the socket.
- 2 Hub validates: authenticated, member of the channel (or valid DM pair, not blocked), not muted or banned, within rate limits, content length and attachment IDs valid.
- 3 Message is persisted with a time-sortable ID (UUIDv7 via `Guid.CreateVersion7()`). The unique constraint on (sender, client message ID) makes retries idempotent.
- 4 Hub broadcasts `MessageCreated` to the channel group (through the Redis backplane on other nodes) and acks the sender with the server ID and timestamp.
- 5 For members not currently connected or subscribed, unread counters are incremented and a notification job is queued (push or email digest).
- 6 After a reconnect the client calls `GET /channels/{id}/messages?after={lastId}` to fill any gap.

Hub contract (v1.0)

Direction	Name	Purpose
Client to server	SubscribeChannel / UnsubscribeChannel	Start or stop receiving events for a channel (membership checked).
Client to server	SendMessage, EditMessage, DeleteMessage	Create, change or soft-delete a message.
Client to server	StartTyping / StopTyping	Ephemeral typing state (auto-expires after 5 s in Redis).
Client to server	MarkRead	Advance the user's last-read message for a channel.
Server to client	MessageCreated / MessageUpdated / MessageDeleted	Message lifecycle events.
Server to client	TypingChanged, PresenceChanged	Ephemeral state updates.
Server to client	UnreadUpdated, MemberJoined, MemberLeft, RoomUpdated, Kicked	Counters and room-level events.

6.4 Media pipeline

- 1 Client calls POST /media/uploads with file name, size and content type. Server checks quota and limits and returns an attachment ID plus a pre-signed upload URL with a random storage key.
- 2 Client uploads directly to object storage (progress bar, cancellable). The API server never proxies file bytes.
- 3 Client calls POST /media/uploads/{id}/complete. Server verifies the object, sniffs the real file type, and queues processing.
- 4 Worker strips EXIF, generates thumbnails or a video poster frame, optionally scans for malware, and marks the attachment **Ready** (or **Rejected**).
- 5 Client sends the message referencing attachment IDs. Only Ready attachments owned by the sender can be attached.
- 6 Media is served through the CDN. Private rooms and DMs use short-lived signed URLs, and non-media files are served as downloads from a separate, cookie-less domain.

6.5 Data model

Design decision: everything is a channel

Rooms contain channels, and a direct conversation is simply a channel of type 'Direct' between two users. Messages, unread state, media, search and reactions therefore work identically in rooms and DMs, and adding multiple channels per room in v1.1 needs no schema change. In v1.0 the UI hides the channel list and shows only #general.

Entity	Key fields
User	Id, Username (normalised, unique), Email (unique), PasswordHash, DisplayName, AvatarKey, Bio, Status, EmailVerifiedAt, CreatedAt, LastSeenAt
RefreshToken	Id, UserId, TokenHash, ExpiresAt, DeviceInfo, RevokedAt, ReplacedById
Room	Id, Name, Description, IconKey, Visibility, OwnerId, MemberCount, Settings (JSON), CreatedAt
RoomMember	RoomId, UserId (composite key), Role, JoinedAt, MutedUntil, NotificationLevel
Channel	Id, Type (RoomText / Direct), RoomId (nullable), Name, Topic, Position, CreatedAt
DirectPair	ChannelId, UserLowId, UserHighId (unique pair, ordered IDs), CreatedAt

Entity	Key fields
Message	Id (UUIDv7), ChannelId, SenderId, Content, ReplyToId, ClientMessageId, CreatedAt, EditedAt, DeletedAt, PinnedAt
Attachment	Id, UploaderId, MessageId (nullable), StorageKey, FileName, ContentType, SizeBytes, Width, Height, DurationSec, ThumbKey, Status
ChannelReadState	UserId, ChannelId (composite key), LastReadMessageId, MentionCount
Reaction	MessageId, UserId, Emoji (composite key), CreatedAt
Invite	Code, RoomId, CreatorId, ExpiresAt, MaxUses, Uses, RevokedAt
RoomBan / UserBlock	RoomId or BlockerId, target UserId, Reason, ActorId, CreatedAt
Report	Id, ReporterId, TargetType, TargetId, Reason, Status, ResolvedBy, CreatedAt
AuditLog	Id, RoomId, ActorId, Action, TargetId, Metadata (JSON), CreatedAt
PushSubscription	Id, UserId, Endpoint, Keys, UserAgent, CreatedAt

Key indexes: Message (ChannelId, Id DESC) for history paging; unique (SenderId, ClientMessageId); GIN index on message text for search; unique on normalised Username and Email; unique on DirectPair (UserLowId, UserHighId); RoomMember (UserId) for 'my rooms'.

6.6 REST API overview

Method	Endpoint	Purpose
POST	/api/auth/register login refresh logout	Account creation and session lifecycle
POST	/api/auth/verify-email forgot-password reset-password	Email verification and recovery
GET / PATCH	/api/users/me	Read and update own profile and settings
GET	/api/users/search?q={prefix}	Find users by username (rate-limited)
GET	/api/users/{username}	Public profile card
POST / GET	/api/rooms /api/rooms/discover	Create room; list my rooms; browse public rooms
GET / PATCH / DELETE	/api/rooms/{roomId}	Room details, settings, deletion
POST	/api/rooms/{roomId}/join leave	Join public room; leave room
GET	/api/rooms/{roomId}/members	Paged member list with roles and presence
PATCH / DELETE	/api/rooms/{roomId}/members/{userId}	Change role; kick
POST / DELETE	/api/rooms/{roomId}/bans	Ban or unban
POST / DELETE	/api/rooms/{roomId}/invites	Create or revoke invite links
POST	/api/invites/{code}/accept	Join through invite link
GET	/api/channels/{channelId}/messages?before= after= limit=	History, paging and gap-fill
GET	/api/channels/{channelId}/messages/search?q=	Full-text search (v1.1)

Method	Endpoint	Purpose
POST / GET	/api/dm	Get-or-create a DM by username; list DM inbox
POST	/api/media/uploads /api/media/uploads/{id}/complete	Media upload handshake
POST / DELETE	/api/blocks/{userId}	Block or unblock a user
POST	/api/reports	Report a message, user or room (v1.1)

All endpoints require a bearer token except registration, login and recovery. Errors use RFC 7807 Problem Details; list endpoints use cursor pagination; the API is versioned under /api/v1 and documented with OpenAPI.

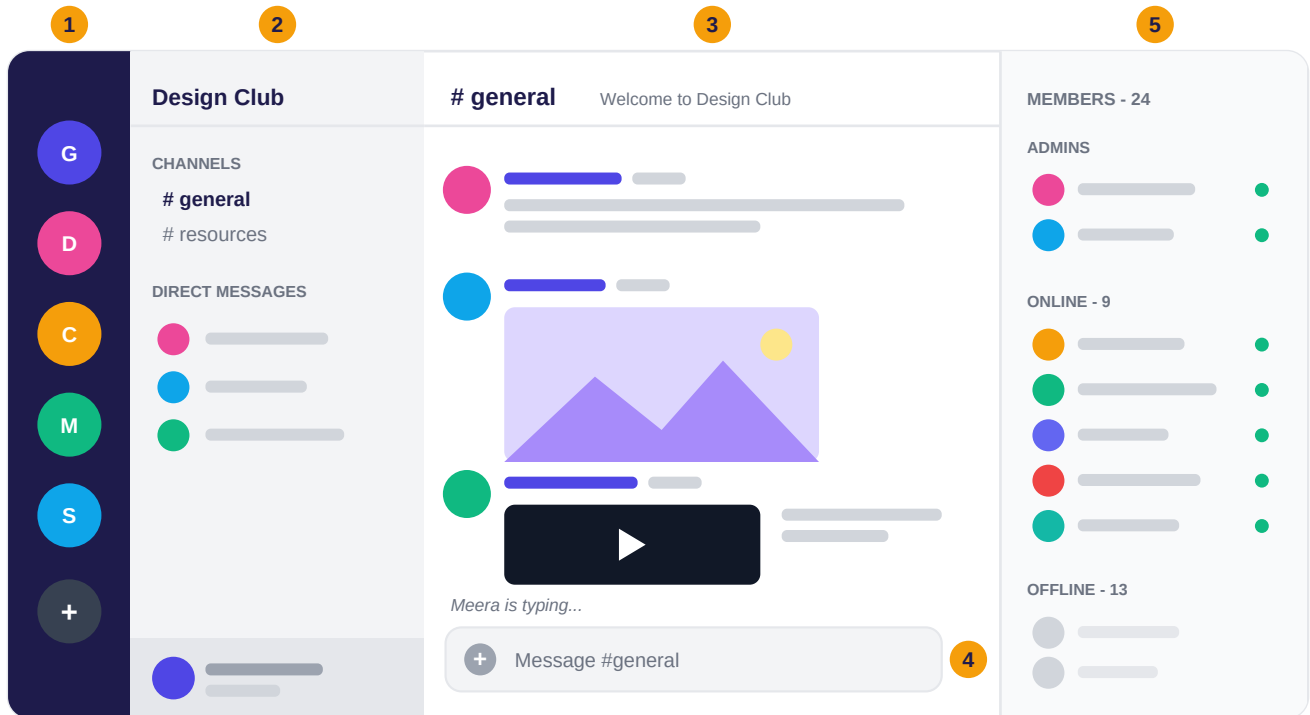
7. Security and Privacy Requirements

- **Authentication:** ASP.NET Core Identity password hashing, logout after repeated failures, refresh-token rotation with reuse detection (revoke the whole token family on reuse). Access token kept in memory only.
- **Authorisation on every path:** each REST endpoint and hub method checks channel membership, role and ban/mute state through resource-based authorisation policies. Never trust IDs sent by the client.
- **Input and output safety:** FluentValidation on all inputs; React escapes output by default, and rendered Markdown is sanitised (DOMPurify or an allow-list renderer). No raw HTML from users.
- **Abuse controls:** ASP.NET Core rate limiter plus Redis counters per user, IP and connection; message throttling per connection; CAPTCHA or proof-of-work on sign-up when abuse is detected.
- **Upload safety:** allow-list of types, magic-byte sniffing, size caps, random storage keys, EXIF removal, malware scanning, separate cookie-less domain for user content, no server-side execution of uploads.
- **Username enumeration and scraping:** minimum query length, rate limits, no bulk listing of users, generic responses on login and recovery.
- **Transport and headers:** TLS everywhere, HSTS, strict CSP, CORS allow-list, secure cookie flags, CSRF protection on cookie-authenticated endpoints.
- **Secrets and infrastructure:** secrets in a managed vault, least-privilege IAM, encrypted databases and storage, dependency and image scanning, regular penetration testing before launch.
- **Privacy:** collect the minimum personal data, provide export and deletion, define retention for messages and media, and document sub-processors (email, push, CDN).

8. UX and Design

8.1 Application layout

Gather uses a familiar four-region layout on desktop that collapses to a single-pane navigation on phones. The wireframe below is illustrative and is meant to align the team before high-fidelity design.



#	Region	Behaviour
1	Room rail	Icons for joined rooms, a DM shortcut at the top, and 'create / discover' at the bottom. Unread dots and mention badges.
2	Sidebar	Room name menu, channel list (hidden in v1.0 with a single channel), recent DMS, and the current user panel with status.
3	Message timeline	Virtualised list with date dividers, grouped messages, image and video previews, hover actions and unread marker.
4	Composer	Multi-line input, attach button, drag-and-drop and paste, emoji picker, upload progress chips, typing indicator above.
5	Member list	Grouped by role and presence; click for profile card with 'Message' action. Collapsible on small screens.

8.2 Screen inventory

Screen	Description	Release
Landing, Sign up, Log in	Value proposition, registration with live username availability check, login, forgot / reset password, verify email.	v1.0
Onboarding	Choose username and avatar, then create a first room or join one through an invite or Discover.	v1.0
Main app shell	Room rail, sidebar, timeline, member list; responsive navigation drawer on mobile.	v1.0

Screen	Description	Release
Room chat	Timeline and composer with media upload, edit and delete, unread marker.	v1.0
DM inbox and chat	List of conversations, new-DM dialog with username search.	v1.0
Create room / Room settings	Overview, Members and roles, Invites, Bans; Audit log and Integrations later.	v1.0
Discover	Browse and search public rooms.	v1.0
Profile card and Settings	Profile, account, privacy (who can DM me), notifications, appearance, blocked users; sessions in v1.1.	v1.0
Media lightbox and gallery	Full-screen image and video viewer; per-room media gallery.	v1.0 / v1.2
Admin console	Reports queue, user and room lookup, takedowns.	v1.1

8.3 Key user flows

First-time user

Landing page, sign up (email, username, password), verify email, pick avatar, then either create a room (name, visibility) and copy the invite link, or paste a received invite. The user lands in a room with a friendly empty state and a prompt to send the first message.

Start a DM by username

Press 'New message' or click a member, type at least three characters of a username, pick the result, and the conversation opens. If a conversation already exists, it opens instead of creating a duplicate. If the recipient's privacy settings block the message, the user sees a clear explanation.

Share images and videos

Drag files into the timeline or paste from the clipboard. Chips show thumbnail, size and progress; the user can add a caption, remove a file or cancel. On send, the message appears immediately with a placeholder that resolves to the processed thumbnail or poster frame.

Join through an invite

The link opens a preview (room name, icon, member count). Logged-out visitors are sent through sign-up or login and return to the same preview; banned users, expired links and full rooms show specific, friendly errors.

8.4 Design principles

- **Fast first:** optimistic UI, skeletons instead of spinners, no layout shift when media loads (reserve space using stored dimensions).
- **Calm by default:** sensible notification defaults, quiet mode, no dark patterns.
- **Keyboard and screen-reader friendly:** every action reachable without a mouse; new messages announced politely.
- **Consistent components:** a shared design system (tokens for colour, spacing, type) with light and dark themes from the start.

9. Success Metrics

Proposed targets for the first 90 days after launch; to be validated against early beta data.

Metric	Definition	Target
Activation	New sign-ups who send at least one message (room or DM) in their first session.	60%
Room activation	Rooms created that have 3 or more members and 10 or more messages within 7 days.	40%
Retention	Users active on day 1 / day 7 / day 30 after sign-up.	40% / 20% / 12%
Engagement	Average messages sent per daily active user.	25 per day
DM adoption	Weekly active users who send at least one DM.	50%
Media sharing	Share of messages with attachments; upload success rate.	10%+ / 98%+
Real-time quality	p95 send-to-receive latency; socket reconnect success within 10 s.	under 300 ms / 99%
Reliability	Crash-free sessions; API error rate (5xx).	99.5% / under 0.5%
Safety	Median time to resolve a report; share of reported content actioned within 24 hours.	under 24 h / 90%

10. Roadmap and Milestones

Indicative plan for a team of about 6 (2 frontend, 2 backend, 1 designer, 1 QA / DevOps shared). Adjust durations to the actual team size.

Phase	Weeks	Deliverables	Exit criteria
0. Discovery and design	1-2	PRD sign-off, design system, wireframes and prototype for core flows, repo, CI/CD, environments.	Design approved; pipeline deploys hello-world to staging.
1. Foundations	3-5	Identity (register, login, verify, reset), profile and avatar, database schema and migrations, app shell, theming.	A user can sign up, log in and edit a profile in staging.
2. Core chat	6-9	Rooms, invites, roles, SignalR messaging, history and pagination, presence, typing, unread state, reconnect and gap-fill.	Two users chat in a room in real time; survives network drops.
3. Media and DMs	10-12	Upload pipeline, image and video sharing, thumbnails, username search, 1:1 DMs, block, privacy setting.	Media works end to end; DM by username works.
4. Hardening and beta	13-15	Moderation tools, rate limiting, in-app notifications, accessibility fixes, security review, load test (10k connections), closed beta.	No P0/P1 defects; load and security targets met.
5. Launch v1.0	16	Public launch, monitoring and on-call, feedback loop.	Go-live checklist complete.
6. v1.1	17-24	Channels, reactions, replies, mentions, search, pins, push and email notifications, reports, friends, PWA.	Engagement and retention targets reviewed.

11. Risks and Mitigations

Risk	Impact	Mitigation
Real-time scale limits (connections, fan-out in large rooms)	High	Redis backplane or Azure SignalR Service from the start; lazy channel subscription; load test early and repeatedly; cap room size in v1.
Harmful or illegal user-uploaded media	High	Upload validation, malware scanning, report flow and fast takedown, ToS and age gate, legal review, evaluate hash-matching or a moderation API for images.
Storage and bandwidth cost from video	Medium	Size caps, per-user quotas, CDN caching, retention policy, later transcoding to smaller renditions.
Duplicate or out-of-order messages	Medium	Server-assigned time-sortable IDs, client message IDs for idempotency, gap-fill on reconnect.
Spam and harassment via username DMs	High	Rate limits, block, privacy settings, message requests, reporting, new-account throttling.
Scope creep towards full Discord parity	Medium	Strict release gating (section 2); every new feature needs a metric and an owner.
Privacy and regulatory compliance	High	Privacy by design, data export and deletion, documented retention, legal review before launch in each region.
Mobile web performance on long timelines	Medium	Virtualised lists, image dimensions stored to avoid layout shift, lazy media, performance budget in CI.

12. Open Questions

#	Question	Recommendation
1	PostgreSQL or SQL Server?	PostgreSQL, unless the organisation already runs SQL Server.
2	Which cloud: Azure or AWS?	Azure gives the smoothest .NET and SignalR Service story; AWS is equally viable. Decide by team skills and credits.
3	Single channel or multi-channel rooms at launch?	Schema supports channels from day one; expose the UI in v1.1.
4	Should strangers be able to DM by username by default?	Default to 'Members of shared rooms' plus message requests once v1.1 lands; allow 'Everyone' as opt-in.
5	Are public rooms discoverable at launch?	Yes, but with an approval or reporting flow to control abuse.
6	Final size limits and media retention policy?	Start with 10 MB images, 100 MB video, 25 MB files; review cost after beta.
7	Can usernames change?	Yes, once every 30 days; keep old names reserved briefly to prevent impersonation.
8	Minimum age and target launch regions?	13+ globally as baseline; confirm with legal for each launch region.
9	Free-tier limits and future monetisation?	Out of scope for v1.0; leave quotas configurable per plan.

Appendix A. Sample Acceptance Criteria

A1. Send a message in a room (MSG-01, MSG-03)

- **Given** I am a member of a room and connected, **when** I send a message, **then** it appears immediately as Pending and becomes Sent within 1 second, and all connected members see it within 300 ms (p95).
- **Given** my connection drops while sending, **when** I tap Retry after reconnecting, **then** the message is delivered exactly once.
- **Given** I am muted or banned in the room, **when** I try to send, **then** the message is rejected with a clear reason and no other member receives it.

A2. Start a DM by username (DM-01, DM-02, DM-04)

- **Given** I type at least 3 characters, **when** matching usernames exist, **then** I see up to 10 results with avatar, display name and username.
- **Given** a conversation with that user already exists, **when** I select them, **then** the existing conversation opens and no duplicate is created.
- **Given** the other user has blocked me, **when** I try to message them, **then** I cannot send, and they are not notified.

A3. Upload and share a video (MEDIA-02, MEDIA-04, MEDIA-06)

- **Given** I select an MP4 under 100 MB, **when** I send it, **then** I see a progress indicator, can cancel, and the message shows a poster frame once processing completes.
- **Given** I rename an executable file to end in .mp4, **when** I upload it, **then** the server rejects it based on content sniffing and shows a clear error.
- **Given** I am not a member of a private room, **when** I request one of its media URLs, **then** access is denied.

A4. Join a room through an invite (ROOM-03)

- **Given** a valid, unexpired invite, **when** a logged-out user opens it, **then** they are guided through sign-up and land in the room.
- **Given** an expired, revoked or max-used invite, **when** it is opened, **then** a specific error is shown and no membership is created.